

LAPORAN HASIL PRAKTIKUM ALGORITMA DAN STRUKTUR DATA
Jobsheet V
BRUTE FORCE DAN DIVIDE CONQUER

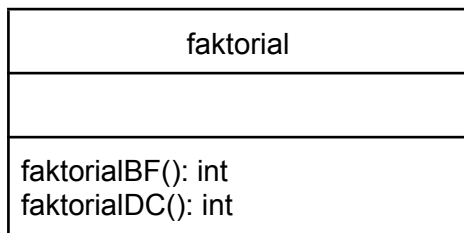


Disusun oleh: Muhammad Toriq Januarsyah
Kelas TI-1H
254107020233

PROGRAM STUDI D-IV TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

5.2 Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer

Perhatikan Diagram Class berikut ini:

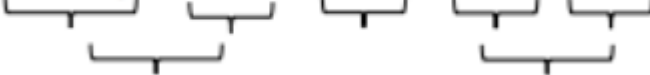


Berdasarkan diagram class di atas, akan dibuat program class dalam Java. Untuk menghitung nilai faktorial suatu angka menggunakan 2 jenis algoritma, Brute Force dan Divide and Conquer. Jika digambarkan terdapat perbedaan proses perhitungan 2 jenis algoritma tersebut sebagai berikut :

Tahapan pencarian nilai faktorial dengan algoritma Brute Force:

$$20! = 20 \times 19 \times 18 \times 17 \times \dots \times 5 \times 4 \times 3 \times 2 \times 1$$


Tahapan pencarian nilai faktorial dengan algoritma Divide and Conquer:

$$20! = 20 \times 19 \times 18 \times 17 \times \dots \times 5 \times 4 \times 3 \times 2 \times 1$$


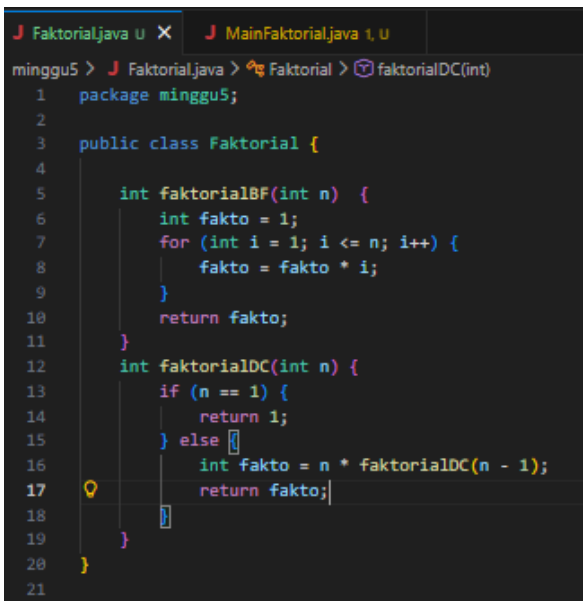
5.2.1 Langkah-langkah Percobaan

1. Buat Project baru, dengan nama "BruteForceDivideConquer". Buat package dengan nama minggu5.
2. Buatlah class baru dengan nama Faktorial
3. Lengkapi class Faktorial dengan atribut dan method yang telah digambarkan di dalam diagram class di atas, sebagai berikut:
 - a) Tambahkan method faktorialBF():

```
int faktorialBF(int n){  
    int fakto = 1;  
    for(int i=1; i<=n; i++){  
        fakto = fakto * i;  
    }  
    return fakto;  
}
```

b) Tambahkan method faktorialDC():

```
int faktorialDC(int n){
    if(n==1){
        return 1;
    }else{
        int fakto = n * faktorialDC(n-1);
        return fakto;
    }
}
```



```
J Faktorial.java U X J MainFaktorial.java t, U
minggu5 > J Faktorial.java > Faktorial > faktorialDC(int)
1 package minggu5;
2
3 public class Faktorial {
4
5     int faktorialBF(int n) {
6         int fakto = 1;
7         for (int i = 1; i <= n; i++) {
8             fakto = fakto * i;
9         }
10        return fakto;
11    }
12    int faktorialDC(int n) {
13        if (n == 1) {
14            return 1;
15        } else {
16            int fakto = n * faktorialDC(n - 1);
17            return fakto;
18        }
19    }
20 }
21
```

4. Coba jalankan (Run) class Faktorial dengan membuat class baru MainFaktorial.

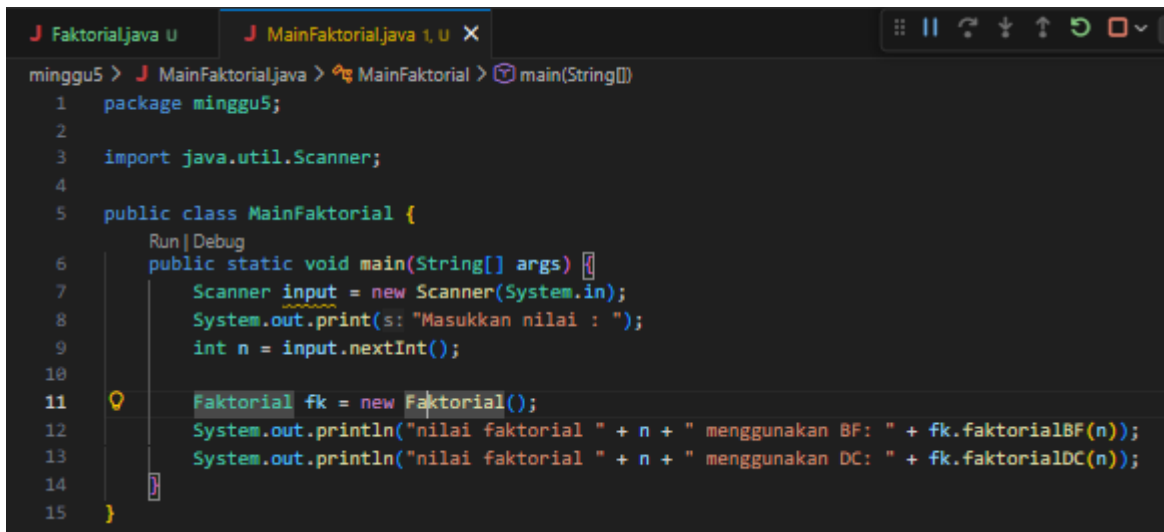
a) Di dalam fungsi main sediakan komunikasi dengan user untuk memasukkan nilai yang akan dicari faktorialnya

```
Scanner input = new Scanner(System.in);
System.out.print(s:"Masukkan nilai: ");
int nilai = input.nextInt();
```

b) Kemudian buat objek dari class Faktorial dan tampilkan hasil pemanggilan method faktorialDC() dan faktorialBF()

```
Faktorial fk = new Faktorial();
System.out.println("Nilai faktorial "+nilai+
    " menggunakan BF: "+fk.faktorialBF(nilai));
System.out.println("Nilai faktorial "+nilai+
    " menggunakan DC: "+fk.faktorialDC(nilai));
```

c) Pastikan program sudah berjalan dengan baik!

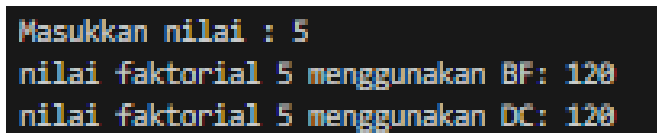


```
minggu5 > J MainFaktorial.java > MainFaktorial > main(String[])
1 package minggu5;
2
3 import java.util.Scanner;
4
5 public class MainFaktorial {
6     public static void main(String[] args) {
7         Scanner input = new Scanner(System.in);
8         System.out.print("Masukkan nilai : ");
9         int n = input.nextInt();
10
11         Faktorial fk = new Faktorial();
12         System.out.println("nilai faktorial " + n + " menggunakan BF: " + fk.faktorialBF(n));
13         System.out.println("nilai faktorial " + n + " menggunakan DC: " + fk.faktorialDC(n));
14     }
15 }
```

5.2.2 Verifikasi Hasil Percobaan

Cocokkan hasil compile kode program anda dengan gambar berikut ini.

```
Masukkan nilai: 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
```



```
Masukkan nilai : 5
nilai faktorial 5 menggunakan BF: 120
nilai faktorial 5 menggunakan DC: 120
```

5.2.3 Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!

Jawab :

- Bagian if:
Berfungsi sebagai base case (kondisi dasar). Pada kode tersebut, if ($n == 1$) digunakan untuk menghentikan rekursi. Ketika nilai n sudah mencapai 1, maka fungsi akan langsung mengembalikan nilai 1 tanpa melakukan pemanggilan fungsi lagi.
- Bagian else:
Berfungsi sebagai recursion call (pemanggilan rekursif). Bagian ini akan terus membagi masalah menjadi sub-masalah yang lebih kecil ($n-1$) dan mengalikannya hingga mendekati base case.

2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!

jawab:

- Ya, sangat memungkinkan. Perulangan for dapat diganti dengan struktur perulangan lain seperti while atau do-while
- Bukti (menggunakan while):

```
1 package minggu5;
2
3 public class Faktorial {
4
5     int faktorialBF(int n) {
6         int fakto = 1;
7         int i = 1;
8         while (i <= n) {
9             fakto = fakto * i;
10            i++;
11        }
12        return fakto;
13    }
14
15    int faktorialDC(int n) {
16        if (n == 1) {
17            return 1;
18        } else {
19            int fakto = n * faktorialDC(n - 1);
20            return fakto;
21        }
22    }
23 }
```

- output :

```
Masukkan nilai : 5
nilai faktorial 5 menggunakan BF: 120
nilai faktorial 5 menggunakan DC: 120
```

3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !

Jawab :

- fakto *= i;
Merupakan pendekatan iteratif (Brute Force). Nilai dihitung secara bertahap di dalam sebuah perulangan dengan mengupdate nilai variabel yang sama berulang kali hingga selesai.
 - int fakto = n * faktorialDC(n-1);
Merupakan pendekatan rekursif (Divide and Conquer). Program memanggil dirinya sendiri dengan parameter yang lebih kecil (n-1). Hasil perkalian baru akan diselesaikan setelah pemanggilan fungsi mencapai kondisi berhenti (base case).
4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

jawab :

- faktorialBF(): Bekerja secara langsung dan berurutan menggunakan perulangan (looping). Ini lebih hemat dalam penggunaan memori stack karena tidak ada penumpukan pemanggilan fungsi.
- faktorialDC(): Bekerja dengan memecah masalah besar menjadi sub-masalah yang lebih kecil melalui rekursi. Cara ini lebih elegan secara penulisan kode (lebih singkat), namun membutuhkan memori lebih besar karena setiap pemanggilan fungsi akan disimpan dalam stack sampai base case terpenuhi.

5.3 Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer

Pada praktikum ini kita akan membuat program class dalam Java, untuk menghitung nilai pangkat suatu angka menggunakan 2 jenis algoritma, Brute Force dan Divide and Conquer. Pada praktikum ini akan digunakan **Array of Object** untuk mengelola beberapa objek yang akan dibuat, berbeda dengan praktikum tahap sebelumnya yang hanya berfokus pada 1 objek factorial saja.

5.3.1 Langkah-langkah Percobaan

1. Di dalam paket minggu5, buatlah class baru dengan nama Pangkat. Dan di dalam class Pangkat tersebut, buat atribut angka yang akan dipangkatkan sekaligus dengan angka pemangkatnya.

```
int nilai, pangkat;
```

2. Tambahkan konstruktor berparameter

```
Pangkat(int n, int p){  
    nilai = n;  
    pangkat = p;  
}
```

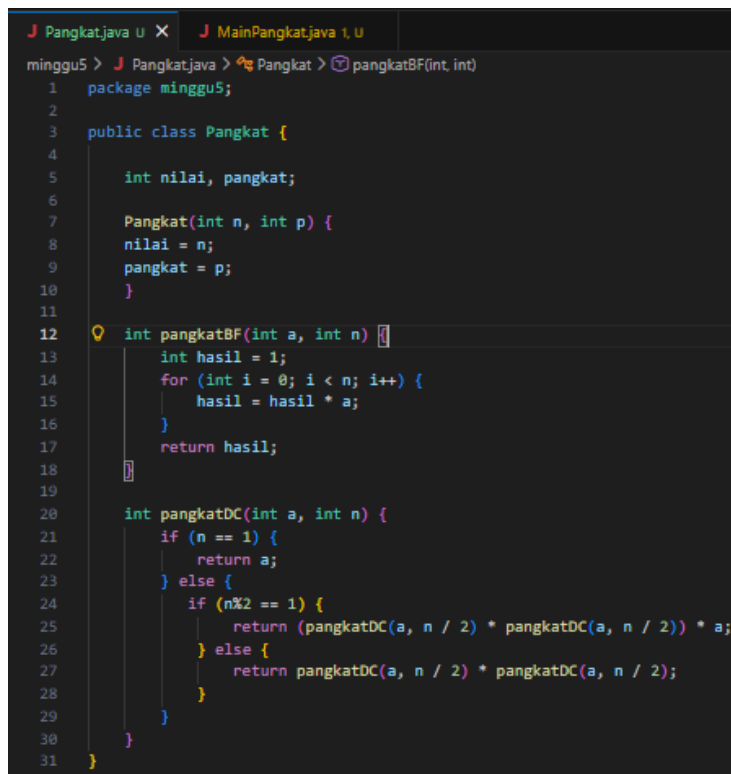
3. Pada class Pangkat tersebut, tambahkan method PangkatBF()

```
int pangkatBF(int a, int n){  
    int hasil = 1;  
    for(int i=0; i<n; i++){  
        hasil = hasil*a;  
    }  
    return hasil;  
}
```

4. Pada class Pangkat juga tambahkan method PangkatDC()

```
int pangkatDC(int a, int n){
    if(n==1){
        return a;
    }else{
        if(n%2==1){
            return (pangkatDC(a, n/2)*pangkatDC(a, n/2)*a);
        }else{
            return (pangkatDC(a, n/2)*pangkatDC(a, n/2));
        }
    }
}
```

5. Perhatikan apakah sudah tidak ada kesalahan yang muncul dalam pembuatan class Pangkat



```
J Pangkat.java x J MainPangkat.java t, u
minggu5 > J Pangkat.java > Pangkat > pangkatBF(int, int)
1 package minggu5;
2
3 public class Pangkat {
4
5     int nilai, pangkat;
6
7     Pangkat(int n, int p) {
8         nilai = n;
9         pangkat = p;
10    }
11
12    int pangkatBF(int a, int n) {
13        int hasil = 1;
14        for (int i = 0; i < n; i++) {
15            hasil = hasil * a;
16        }
17        return hasil;
18    }
19
20    int pangkatDC(int a, int n) {
21        if (n == 1) {
22            return a;
23        } else {
24            if (n%2 == 1) {
25                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2)) * a;
26            } else {
27                return pangkatDC(a, n / 2) * pangkatDC(a, n / 2);
28            }
29        }
30    }
31 }
```

6. Selanjutnya buat class baru yang di dalamnya terdapat method main. Class tersebut dapat dinamakan MainPangkat. Tambahkan kode pada class main untuk menginputkan jumlah elemen yang akan dihitung pangkatnya.

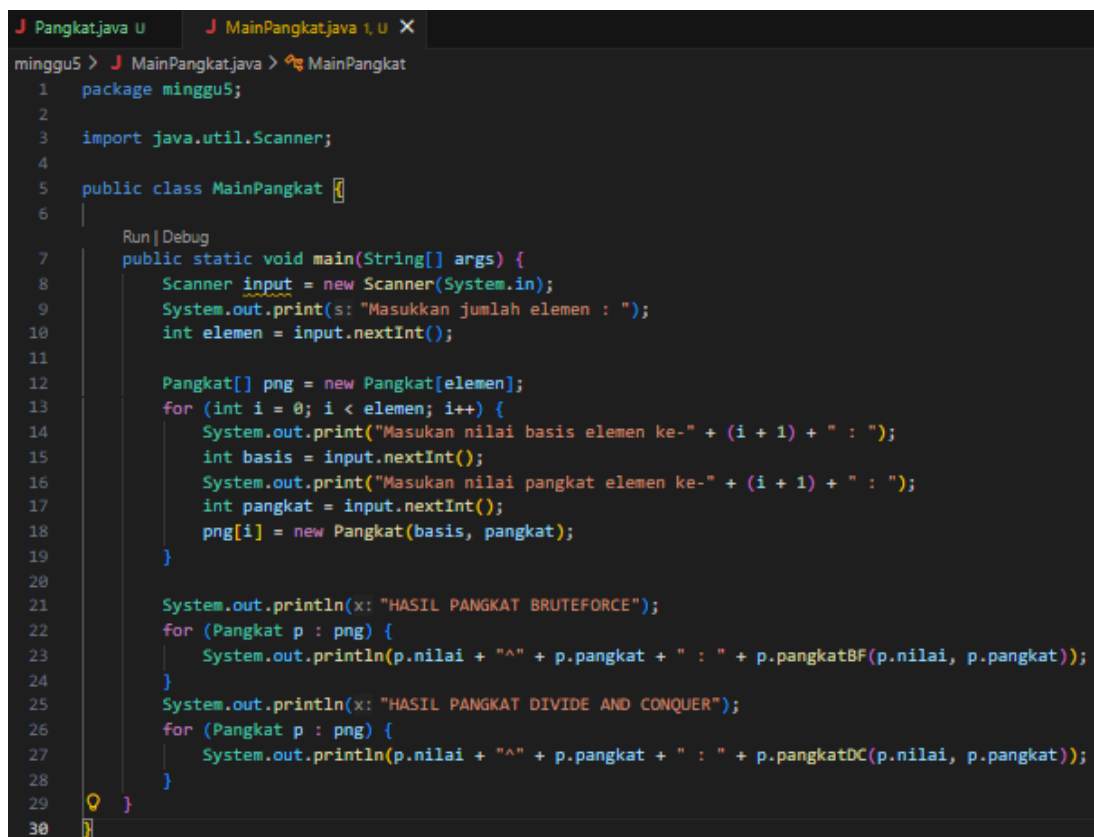
```
Scanner input = new Scanner(System.in);
System.out.print(s:"Masukkan jumlah elemen: ");
int elemen = input.nextInt();
```

7. Nilai pada tahap 5 selanjutnya digunakan untuk instansiasi array of objek. Di dalam Kode berikut ditambahkan proses pengisian beberapa nilai yang akan dipangkatkan sekaligus dengan pemangkatnya.

```
Pangkat[] png = new Pangkat[elemen];
for(int i=0;i<elemen;i++){
    System.out.print("Masukan nilai basis elemen ke-"+(i+1)+": ");
    int basis = input.nextInt();
    System.out.print("Masukan nilai pangkat elemen ke-"+(i+1)+": ");
    int pangkat = input.nextInt();
    png[i] = new Pangkat(basis, pangkat);
}
```

8. Kemudian, panggil hasil nya dengan mengeluarkan return value dari method PangkatBF() dan PangkatDC().

```
System.out.println(x:"HASIL PANGKAT BRUTEFORCE:");
for (Pangkat p : png) {
    System.out.println(p.nilai+"^"+p.pangkat+": "+p.pangkatBF(p.nilai, p.pangkat));
}
System.out.println(x:"HASIL PANGKAT DIVIDE AND CONQUER:");
for (Pangkat p : png) {
    System.out.println(p.nilai+"^"+p.pangkat+": "+p.pangkatDC(p.nilai, p.pangkat));
}
```



```
J Pangkat.java U    J MainPangkat.java 1, U X
minggu5 > J MainPangkat.java > MainPangkat
1  package minggu5;
2
3  import java.util.Scanner;
4
5  public class MainPangkat {
6
7      Run | Debug
8      public static void main(String[] args) {
9          Scanner input = new Scanner(System.in);
10         System.out.print(s: "Masukkan jumlah elemen : ");
11         int elemen = input.nextInt();
12
13         Pangkat[] png = new Pangkat[elemen];
14         for (int i = 0; i < elemen; i++) {
15             System.out.print("Masukan nilai basis elemen ke- " + (i + 1) + " : ");
16             int basis = input.nextInt();
17             System.out.print("Masukan nilai pangkat elemen ke- " + (i + 1) + " : ");
18             int pangkat = input.nextInt();
19             png[i] = new Pangkat(basis, pangkat);
20         }
21
22         System.out.println(x: "HASIL PANGKAT BRUTEFORCE");
23         for (Pangkat p : png) {
24             System.out.println(p.nilai + "^" + p.pangkat + " : " + p.pangkatBF(p.nilai, p.pangkat));
25         }
26         System.out.println(x: "HASIL PANGKAT DIVIDE AND CONQUER");
27         for (Pangkat p : png) {
28             System.out.println(p.nilai + "^" + p.pangkat + " : " + p.pangkatDC(p.nilai, p.pangkat));
29         }
30     }
```

5.3.2 Verifikasi Hasil Percobaan

Pastikan output yang ditampilkan sudah benar seperti di bawah ini.

```
Masukkan jumlah elemen: 3
Masukan nilai basis elemen ke-1: 2
Masukan nilai pangkat elemen ke-1: 3
Masukan nilai basis elemen ke-2: 4
Masukan nilai pangkat elemen ke-2: 5
Masukan nilai basis elemen ke-3: 6
Masukan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
```

```
Masukkan jumlah elemen : 3
Masukan nilai basis elemen ke-1 : 2
Masukan nilai pangkat elemen ke-1 : 3
Masukan nilai basis elemen ke-2 : 4
Masukan nilai pangkat elemen ke-2 : 5
Masukan nilai basis elemen ke-3 : 6
Masukan nilai pangkat elemen ke-3 : 7
HASIL PANGKAT BRUTEFORCE
2^3 : 8
4^5 : 1024
6^7 : 279936
HASIL PANGKAT DIVIDE AND CONQUER
2^3 : 8
4^5 : 1024
6^7 : 279936
```

5.3.3 Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!

Jawab :

- pangkatBF()

Menggunakan algoritma Brute Force dengan prinsip iterasi (perulangan). Cara kerjanya adalah mengalikan bilangan basis secara berulang sebanyak n kali sesuai jumlah pangkatnya.

- pangkatDC()

Menggunakan algoritma Divide and Conquer dengan prinsip rekursif. Cara kerjanya adalah memecah pangkat menjadi dua bagian (setengahnya) secara terus-menerus hingga mencapai kondisi dasar (base case), sehingga jumlah operasi perkalian yang dilakukan jauh lebih sedikit dan lebih efisien.

2. Apakah tahap combine sudah termasuk dalam kode tersebut? Tunjukkan!

jawab :

- Ya, sudah termasuk. Tahap combine (penggabungan) terjadi saat hasil dari pemanggilan rekursif dikalikan kembali.
- Tunjukkan: Pada potongan kode di dalam blok else, yaitu:

`return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);` (untuk ganjil) atau

`return (pangkatDC(a, n/2) * pangkatDC(a, n/2));` (untuk genap).

Operasi perkalian `*` itulah yang berfungsi menggabungkan hasil-hasil dari sub-masalah yang telah dipecah.

3. Pada method pangkatBF() terdapat parameter untuk melewati nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?

- Tetap relevan dan Bisa. Method dapat langsung mengakses atribut nilai dan pangkat yang ada di dalam class tersebut menggunakan kata kunci `this`.

contoh method `pangkatBF()` tanpa parameter

```
int pangkatBF() {
    int hasil = 1;
    for (int i = 0; i < this.pangkat; i++) {
        hasil = hasil * this.nilai;
    }
    return hasil;
}
```

4. Tarik tentang cara kerja method `pangkatBF()` dan `pangkatDC()`!
 - `pangkatBF()` bekerja secara linear. Jika pangkatnya adalah 1000, maka akan terjadi 1000 kali perulangan. Ini mudah diimplementasikan tetapi lambat untuk angka yang sangat besar.
 - `pangkatDC()` bekerja secara logaritmik. Dengan membagi masalah menjadi dua bagian di setiap tahap, jumlah operasi berkurang drastis. Cara ini jauh lebih optimal untuk menangani perhitungan kompleks atau angka yang sangat besar.

5.4 Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer

Di dalam percobaan ini, kita akan mempraktekkan bagaimana proses *divide*, *conquer*, dan *combine* diterapkan pada studi kasus penjumlahan keuntungan suatu perusahaan dalam beberapa bulan.

5.4.1 Langkah-langkah Percobaan

1. Pada paket **minggu5**. Buat class baru yaitu class `Sum`. Tambahkan pula konstruktor pada class `Sum`.

```
double keuntungan[];

Sum(int el){
    keuntungan = new double[el];
}
```

2. Tambahkan method `TotalBF()` yang akan menghitung total nilai array dengan cara iterative.

```
double totalBF(){
    double total=0;
    for(int i=0;i<keuntungan.length;i++){
        total = total+keuntungan[i];
    }
    return total;
}
```

3. Tambahkan pula method `TotalDC()` untuk implementasi perhitungan nilai total array menggunakan algoritma Divide and Conquer

```
double totalDC(double arr[], int l, int r){
    if(l==r){
        return arr[l];
    }

    int mid = (l+r)/2;
    double lsum = totalDC(arr, l, mid);
    double rsum = totalDC(arr, mid+1, r);
    return lsum+rsum;
}
```

```
J Sum.java U X J MainSum.java t, U
minggu5 > J Sum.java > Sum > totalBF()
1 package minggu5;
2
3 public class Sum {
4     double keuntungan[];
5
6     Sum (int e1) {
7         keuntungan = new double[e1];
8     }
9
10    double totalBF() {
11        double total = 0;
12        for (int i = 0; i < keuntungan.length; i++) {
13            total = total + keuntungan[i];
14        }
15        return total;
16    }
17
18    double totalDC(double arr[], int l, int r) {
19        if (l == r) {
20            return arr[l];
21        }
22
23        int mid = (l + r) / 2;
24        double lsum = totalDC(arr, l, mid);
25        double rsum = totalDC(arr, mid + 1, r);
26        return lsum + rsum;
27    }
28 }
```

4. Buat class baru yaitu MainSum. Di dalam kelas ini terdapat method main. Pada method ini user dapat menuliskan berapa bulan keuntungan yang akan dihitung. Dalam kelas ini sekaligus dibuat instansiasi objek untuk memanggil atribut ataupun fungsi pada class Sum

```
Scanner input = new Scanner(System.in);
System.out.print(s:"Masukkan jumlah elemen: ");
int elemen = input.nextInt();
```

5. Buat objek dari class Sum. Lakukan perulangan untuk mengambil input nilai keuntungan dan masukkan ke atribut keuntungan dari objek yang baru dibuat tersebut!

```
Sum sm = new Sum(elemen);
for(int i=0;i<elemen;i++){
    System.out.print("Masukkan keuntungan ke-"+(i+1)+" : ");
    sm.keuntungan[i] = input.nextDouble();
}
```

6. Tampilkan hasil perhitungan melalui objek yang telah dibuat untuk kedua cara yang ada (Brute Force dan Divide and Conquer)

```
System.out.println("Total keuntungan menggunakan Bruteforce: "+sm.totalBF());
System.out.println("Total keuntungan menggunakan Divide and Conquer: "+sm.totalDC(sm.keuntungan,1:0,elemen-1))
```



```
J Sum.java U J MainSum.java 1, U X
minggu5 > J MainSum.java > ...
1 package minggu5;
2
3 import java.util.Scanner;
4
5 public class MainSum {
6     Run | Debug
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         System.out.print(s: "Masukkan jumlah elemen : ");
10        int elemen = sc.nextInt();
11        Sum sm = new Sum(elemen);
12        for (int i = 0; i < elemen; i++) {
13            System.out.print("Masukkan keuntungan bulan ke- " + (i + 1) + " : ");
14            sm.keuntungan[i] = sc.nextDouble();
15        }
16
17        System.out.println("Total keuntungan dengan Brute Force: " + sm.totalBF());
18        System.out.println("Total keuntungan dengan Divide and Conquer: " + sm.totalDC(sm.keuntungan, 1: 0, sm.keuntungan.length - 1));
19    }
}
```

5.4.2 Verifikasi Hasil Percobaan

Cocokkan hasil compile kode program anda dengan gambar berikut ini.

```
Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0
```

```
Masukkan jumlah elemen : 5
Masukkan keuntungan bulan ke-1 : 10
Masukkan keuntungan bulan ke-2 : 20
Masukkan keuntungan bulan ke-3 : 30
Masukkan keuntungan bulan ke-4 : 40
Masukkan keuntungan bulan ke-5 : 50
Total keuntungan dengan Brute Force: 150.0
Total keuntungan dengan Divide and Conquer: 150.0
```

5.4.3 Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?

jawab :

- karena berfungsi sebagai titik tengah atau pembatas untuk membagi (divide) barisan data array menjadi dua bagian, yaitu bagian kiri (dari indeks l sampai mid) dan bagian kanan (dari indeks mid+1 sampai r). Tanpa mid, kita tidak bisa memecah masalah besar menjadi sub-masalah yang lebih kecil secara konsisten.

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```
double lsum = totalDC(arr, l, mid);  
double rsum = totalDC(arr, mid+1, r);
```

jawab :

- tsatement tersebut merupakan Base Case (kondisi berhenti). Program mengecek apakah indeks kiri (l) sudah sama dengan indeks kanan (r). Jika sudah sama, artinya array tersebut sudah terbagi hingga tersisa satu elemen saja, sehingga nilai elemen tersebut bisa langsung dikembalikan (return) tanpa perlu dibagi lagi.

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

jawab :

- Penjumlahan tersebut merupakan tahap Combine dalam strategi Divide and Conquer. Setelah array dipecah-pecah dan nilai terkecilnya ditemukan, kita harus menjumlahkan kembali hasil dari bagian kiri (lsum) dan hasil dari bagian kanan (rsum) untuk mendapatkan total keseluruhan dari rentang array tersebut.

4. Apakah base case dari totalDC()?

jawab :

- Base case dari TotalDC() adalah ketika kondisi if (l == r) terpenuhi. Pada kondisi ini, rentang data yang diproses hanya mencakup satu elemen saja, sehingga fungsi akan mengembalikan nilai dari elemen tersebut: return arr[l];

5. Tarik Kesimpulan tentang cara kerja totalDC()

jawab :

- TotalDC() bekerja dengan cara membagi array keuntungan secara rekursif menjadi dua bagian terus-menerus hingga mencapai elemen tunggal (divide). Setelah mencapai elemen tunggal, program akan mulai menjumlahkan nilai-nilai tersebut secara bertahap dari bawah ke atas (combine) hingga kembali ke pemanggilan awal, sehingga didapatkan total keseluruhan nilai array dengan lebih terstruktur.

5.5 Latihan Praktikum

1. Sebuah kampus memiliki daftar nilai mahasiswa dengan data sesuai tabel di bawah ini

Nama	NIM	Tahun Masuk	Nilai UTS	Nilai UAS
Ahmad	220101001	2022	78	82
Budi	220101002	2022	85	88
Cindy	220101003	2021	90	87
Dian	220101004	2021	76	79
Eko	220101005	2023	92	95
Fajar	220101006	2020	88	85
Gina	220101007	2023	80	83
Hadi	220101008	2020	82	84

Tentukan :

- a) Nilai UTS tertinggi tertinggi menggunakan Divide and Conquer!
- b) Nilai UTS terendah menggunakan Divide and Conquer!
- c) Rata-rata nilai UAS dari semua mahasiswa menggunakan Brute Force!

jawab :

kode program : class mahasiswa

```
J Mahasiswajava u X J MainMahasiswajava u
minggu5 > J Mahasiswajava > Mahasiswa
1 package minggu5;
2
3 public class Mahasiswa {
4     public String nama;
5     public String nim;
6     public int tahunMasuk;
7     public double nilaiUTS;
8     public double nilaiUAS;
9
10    public Mahasiswa(String nama, String nim, int tahun, double uts, double uas) {
11        this.nama = nama;
12        this.nim = nim;
13        this.tahunMasuk = tahun;
14        this.nilaiUTS = uts;
15        this.nilaiUAS = uas;
16    }
17
18    public static Mahasiswa cariUTStertinggi(Mahasiswa arr[], int l, int r) {
19        if (l == r) {
20            return arr[l];
21        }
22        int mid = (l + r) / 2;
23        Mahasiswa kiri = cariUTStertinggi(arr, l, mid);
24        Mahasiswa kanan = cariUTStertinggi(arr, mid + 1, r);
25
26        return (kiri.nilaiUTS > kanan.nilaiUTS) ? kiri : kanan;
27    }
28
29    public static Mahasiswa cariUTSterendah(Mahasiswa arr[], int l, int r) {
30        if (l == r) {
31            return arr[l];
32        }
33        int mid = (l + r) / 2;
34        Mahasiswa kiri = cariUTSterendah(arr, l, mid);
35        Mahasiswa kanan = cariUTSterendah(arr, mid + 1, r);
36
37        return (kiri.nilaiUTS < kanan.nilaiUTS) ? kiri : kanan;
38    }
39
40    public static double hitungRataUAS(Mahasiswa arr[]) {
41        double total = 0;
42        for (int i = 0; i < arr.length; i++) {
43            total += arr[i].nilaiUAS;
44        }
45        return total / arr.length;
46    }
47 }
```

kode program : MainMahasiswa

```
J Mahasiswa.java U J MainMahasiswa.java U X
minggu5 > J MainMahasiswa.java > MainMahasiswa > main(String[])
1 package minggu5;
2
3 public class MainMahasiswa {
4     Run | Debug
5     public static void main(String[] args) {
6
7         Mahasiswa[] daftarMhs = {
8             new Mahasiswa(nama: "Ahmad", nim: "220101001", tahun: 2022, uts: 78, uas: 82),
9             new Mahasiswa(nama: "Budi", nim: "220101002", tahun: 2022, uts: 85, uas: 88),
10            new Mahasiswa(nama: "Cindy", nim: "220101003", tahun: 2021, uts: 90, uas: 87),
11            new Mahasiswa(nama: "Dian", nim: "220101004", tahun: 2021, uts: 76, uas: 79),
12            new Mahasiswa(nama: "Eko", nim: "220101005", tahun: 2023, uts: 92, uas: 95),
13            new Mahasiswa(nama: "Fajar", nim: "220101006", tahun: 2020, uts: 88, uas: 85),
14            new Mahasiswa(nama: "Gina", nim: "220101007", tahun: 2023, uts: 80, uas: 83),
15            new Mahasiswa(nama: "Hadi", nim: "220101008", tahun: 2020, uts: 82, uas: 84)
16        };
17
18        Mahasiswa mhsTertinggi = Mahasiswa.cariUTSTertinggi(daftarMhs, 1: 0, daftarMhs.length - 1);
19
20        Mahasiswa mhsTerendah = Mahasiswa.cariUTSTerendah(daftarMhs, 1: 0, daftarMhs.length - 1);
21
22        double rataUAS = Mahasiswa.hitungRataUAS(daftarMhs);
23
24        System.out.println(x: "=====");
25        System.out.println(x: "HASIL ANALISIS DATA MAHASISWA");
26        System.out.println(x: "=====");
27
28        System.out.println(x: "A. MAHASISWA DENGAN UTS TERTINGGI (DC):");
29        System.out.println(" Nama : " + mhsTertinggi.nama);
30        System.out.println(" NIM : " + mhsTertinggi.nim);
31        System.out.println(" Tahun Masuk : " + mhsTertinggi.tahunMasuk);
32        System.out.println(" Nilai UTS : " + mhsTertinggi.nilaiUTS);
33
34        System.out.println(x: "\nB. MAHASISWA DENGAN UTS TERENDAH (DC):");
35        System.out.println(" Nama : " + mhsTerendah.nama);
36        System.out.println(" NIM : " + mhsTerendah.nim);
37        System.out.println(" Tahun Masuk : " + mhsTerendah.tahunMasuk);
38        System.out.println(" Nilai UTS : " + mhsTerendah.nilaiUTS);
39
40        System.out.printf(format: "\nC. RATA-RATA NILAI UAS (BRUTE FORCE): %.2f\n", rataUAS);
41        System.out.println(x: "=====");
42    }
43 }
```

output :

```
=====
HASIL ANALISIS DATA MAHASISWA
=====
A. MAHASISWA DENGAN UTS TERTINGGI (DC):
Nama      : Eko
NIM       : 220101005
Tahun Masuk : 2023
Nilai UTS  : 92.0

B. MAHASISWA DENGAN UTS TERENDAH (DC):
Nama      : Dian
NIM       : 220101004
Tahun Masuk : 2021
Nilai UTS  : 76.0

C. RATA-RATA NILAI UAS (BRUTE FORCE): 85.38
=====
```